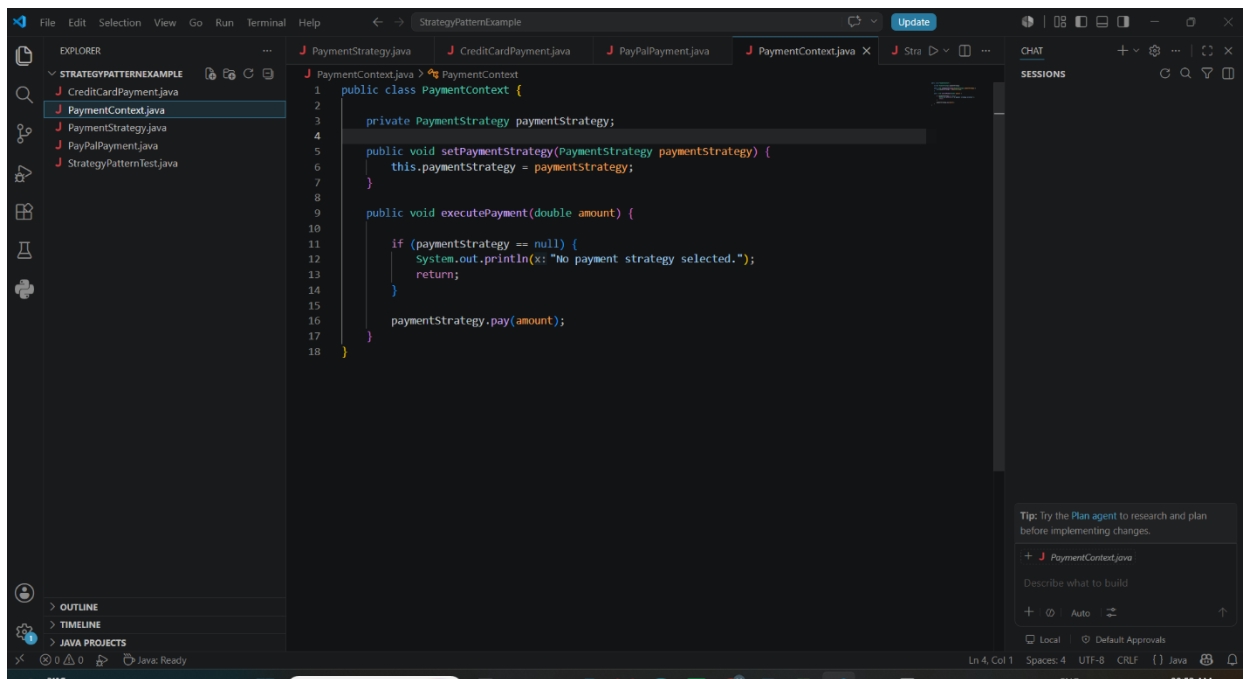


Hands-On

Exercise 8: Implementing the Strategy Pattern

As part of this module, I successfully implemented, executed, and tested the Strategy Pattern in Java by developing a project named StrategyPatternExample. I created a PaymentStrategy interface with a pay() method and implemented concrete strategy classes such as CreditCardPayment and PayPalPayment, each providing its own payment processing logic. I also developed a PaymentContext class that maintained a reference to a PaymentStrategy object and executed the selected payment strategy at runtime. After completing the implementation, I ran the program and tested different payment methods by dynamically switching between strategies to verify their functionality. This hands-on exercise helped me understand how the Strategy Pattern enables the selection of algorithms or behaviors at runtime, promotes flexibility, reduces conditional logic, and makes applications easier to extend and maintain.

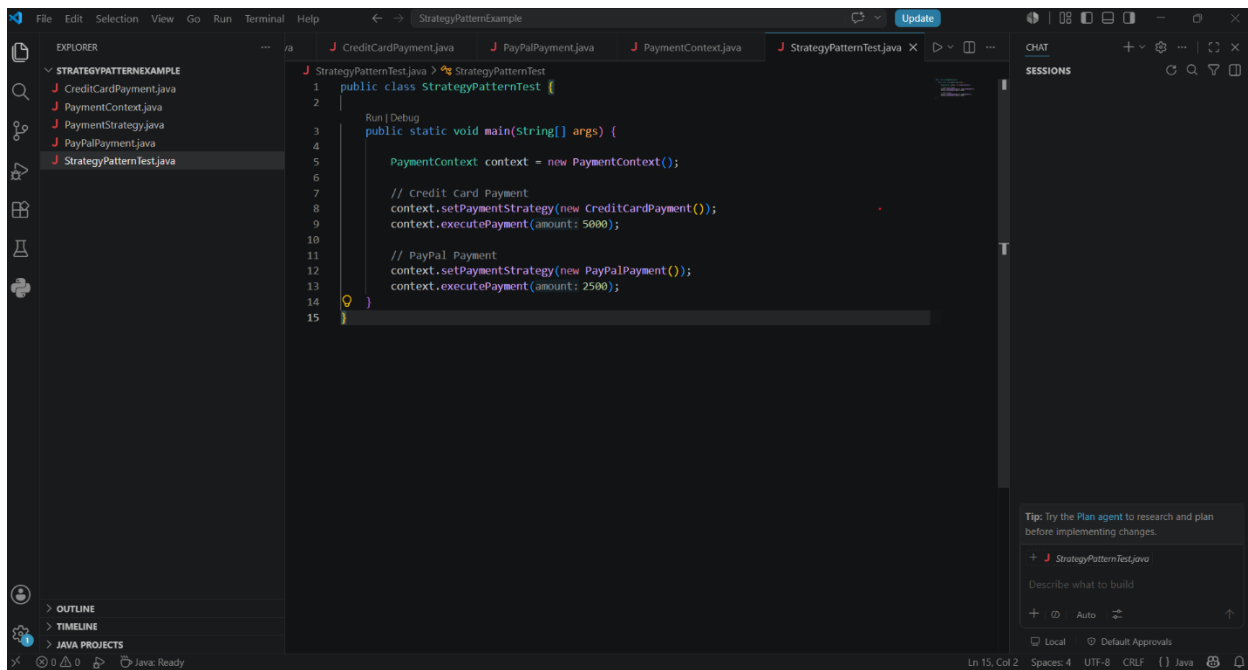
Coding:



The screenshot shows an IDE window titled "StrategyPatternExample" with several open files. The Explorer panel on the left shows the project structure: "STRATEGYPATTERNEXAMPLE" containing "CreditCardPayment.java", "PaymentContext.java", "PaymentStrategy.java", "PayPalPayment.java", and "StrategyPatternTest.java". The main editor displays the code for "PaymentContext.java".

```
1 public class PaymentContext {
2
3     private PaymentStrategy paymentStrategy;
4
5     public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
6         this.paymentStrategy = paymentStrategy;
7     }
8
9     public void executePayment(double amount) {
10
11         if (paymentStrategy == null) {
12             System.out.println(x: "No payment strategy selected.");
13             return;
14         }
15
16         paymentStrategy.pay(amount);
17     }
18 }
```

The bottom right of the IDE shows a chat window with a tip: "Tip: Try the Plan agent to research and plan before implementing changes." and a button to "+ PaymentContext.java". The status bar at the bottom indicates "Ln 4, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "Java".



Output:

